Listen

Learning Doc 06: The Sync Protocol
Keeping a laptop and a server in agreement, cheaply and offline-first

Detailed design, Section 5

13 October 2026

Abstract

Your laptop and the cloud each hold a copy of some data. This document designs how those copies stay in agreement: what is sent, when, in which direction, what happens when you are offline, when the same message arrives twice, when you delete something, and when you edit the same command on both sides. The whole exchange is one small request every five minutes.

Contents

1 Decisions this section is built on **2**

2 Who talks to whom **2**

3 One round trip every 5 minutes **2**

4 Uploading history: safe even when messages repeat **3**

5 Deleting: tombstones **4**

6 Commands: two editors, one master **4**
6.1 Laptop → server: snapshots 4
6.2 Server → laptop: edits wait to be applied 5

7 Who proves it is really your laptop **6**

8 When things go wrong **6**

9 Cost of all this **6**

10 Glossary **6**

11 Check your understanding **7**

1 Decisions this section is built on

Topic	Decision
Command files	The YAML files on the laptop are the master, kept in the repository's `commands/` folder (versioned in git). They can be edited in PyCharm *or* in the web app; web edits are written into the YAML.
Reconnecting	Sync immediately when Windows reports the network is back.
Conflicts	If both sides changed the same command, the laptop version wins.
Frequency	Every 5 minutes. Lightweight first. The dashboard refreshes every 5 minutes too.
Deleting history	From the review list and the web app (no voice). A deletion on one side reaches the other.
Privacy (Doc 05)	Only syncable tables are ever read by the sync code.

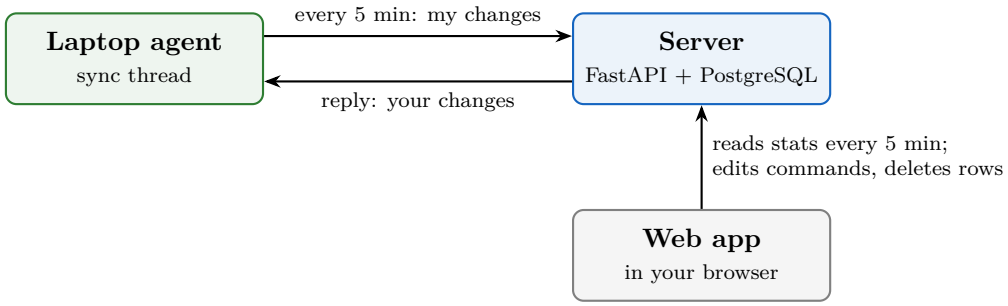
2 Who talks to whom

Concept: Polling

The server cannot contact your laptop: your laptop sits behind home Wi-Fi with no public address, and may be off. So the laptop **polls**: every 5 minutes *it* contacts the server, sends what is new, and asks "anything new for me?". The server only ever answers.

Analogy

A post office box. The post office cannot deliver to a house it cannot find, so it keeps your mail in a box. Every so often you visit, drop off your outgoing letters and collect what is waiting. If you are away for a week, the mail simply waits.



3 One round trip every 5 minutes

Each sync is a **single HTTPS request** carrying everything the laptop has to say, and a single reply carrying everything the server has to say. One request instead of five separate ones keeps the network use and battery cost close to zero.

```
POST /sync                                    (laptop -> server)
{
  "cursor": 1841,                    # "I have seen server changes up to #1841"
  "rows":      [ ...new or changed commands / usage_daily rows... ],
  "deletions": [ "uuid-1", "uuid-2" ],          # rows you deleted locally
  "commands_snapshot": { "hash": "a91f...", "files": {...} },  # only if changed
  "edit_results": [ {"edit": 17, "status": "applied"} ]
}
```

```
reply                                          (server -> laptop)
{
  "accepted":  [ "uuid-...", ... ],           # rows the server stored
  "deletions": [ "uuid-9" ],                   # rows you deleted in the web app
  "pending_edits": [ {...an edit made in the web app...} ],
  "cursor": 1856
}
```

When nothing has changed, the request is a few hundred bytes. When the laptop is offline, nothing is lost: unsent rows simply stay unsent until the next successful sync.

4 Uploading history: safe even when messages repeat

What needs sending. A row in a syncable table needs uploading if it was never sent (`synced_at` is empty) or it changed since it was sent (`updated_at` > `synced_at`). The second case matters: when you mark a command wrong, or today's usage total grows, the row changes *after* it was uploaded.

Batches. At most 500 rows per request; if there are more (after a week offline), the next sync continues immediately until the backlog is gone. Why a limit at all? One huge request after a long offline stretch would (1) lose *all* progress if the connection drops near the end, (2) risk hitting network and server time limits on slow Wi-Fi, and (3) need the whole backlog in memory on both sides. With batches, every confirmed batch is permanently done. Why 500: a row is about 0.5–1 KB, so a batch is roughly 250–500 KB, under a second on ordinary internet. Fifty would mean dozens of round trips for a week's backlog; five thousand brings back the problems above. It is a tunable starting value, and in normal use (0–20 rows per sync) it never comes into play.

Concept: Outbox pattern (store and forward)

There is no separate queue of "messages to send". The unsent data *is* the rows themselves, sitting in SQLite with `synced_at` empty or older than `updated_at`; "what to send" is just a query. Nothing expires: rows wait for days or months if needed, and are marked sent only after the server confirms them. The 30-day purge only touches the private tables, which are never in the outbox at all.

Concept: Idempotency

An operation is **idempotent** if doing it twice has the same effect as doing it once. Networks fail in an awkward way: the server may store your rows and then the reply is lost, so the laptop thinks it failed and sends them again. Because every row carries its `uuid`, the server performs an **upsert** ("insert, or update if that uuid already exists"), so receiving the same row twice changes nothing. That lets the laptop simply retry anything that was not confirmed.

After the reply arrives, the laptop sets `synced_at` only on the uuids the server listed as `accepted`.

Concept: Retry with back-off

If a sync fails (no internet, server down), the next attempt waits longer: 5, 10, 20, then at most 30 minutes. This is **exponential back-off**: it stops a laptop with no Wi-Fi from wasting effort retrying every few seconds. One success resets it to 5 minutes.

Back online: sync immediately (decided). Waiting up to 30 minutes after the internet returns is unnecessary. Windows announces network changes; the sync thread subscribes to that notification

and, when a connection appears, syncs at once instead of waiting for the back-off timer. Subscribing costs nothing while waiting: Windows wakes the thread only when the network actually changes.

5 Deleting: tombstones

If you delete a row locally and the laptop just forgets it, the next sync has nothing to tell the server, and the server keeps the row forever. Deletions must be *remembered* until they have been delivered.

Concept: Tombstone

A **tombstone** is a marker that says "this row was deleted". Instead of removing the row immediately, Listen sets its `deleted_at`. It disappears from every list at once, but it still exists long enough for the sync to report its uuid. Once the other side confirms, the row is removed for real.

- **Deleted in the review list:** tombstone locally → next sync sends the uuid → server deletes its copy → laptop removes the row.
- **Deleted in the web app:** server removes it and records the deletion in its change log → next sync returns the uuid → laptop removes its copy.

The resurrection problem, and the server's tombstones. A tombstone is crossing an entry out of a shared notebook instead of tearing the page out: anyone reading later still learns it was deleted. Consider this sequence:

1. Monday: a command syncs to the server.
2. Tuesday: you delete it in the web app; the server removes it.
3. Tuesday evening, offline: you mark that same command "wrong" in the review list, so its `updated_at` changes and the laptop wants to re-upload it.
4. Wednesday: the upload arrives; an upsert finds no such row and *inserts it again*. Your deleted command is back.

To prevent this, the server also keeps a tombstone (the uuid and the deletion time, about 40 bytes, kept indefinitely). Any upload of a tombstoned uuid is refused, and the laptop removes its copy when it receives the deletion.

Concept: Cursor (change log)

The server numbers every change it makes ("change #1856: row uuid-9 deleted"). The laptop remembers the last number it has seen, the **cursor**. Each sync asks for "everything after #1841". Nothing is missed, nothing is sent twice, and the server does not have to remember which laptop has seen what: the laptop keeps its own place, like a bookmark.

Where it lives: the numbered change log is on the server; the cursor is on the laptop, in the `meta` table. Like a book: the publisher prints the page numbers, each reader keeps a bookmark. A second device would simply have its own bookmark, and if the laptop's database were lost, the cursor restarts at 0 and the laptop re-reads every change; nothing breaks.

6 Commands: two editors, one master

Command files are different from history. History only ever flows *up*. Commands can be edited on *both* sides, and only one copy can be the truth.

6.1 Laptop → server: snapshots

Whenever the command files change (you saved in PyCharm, or a web edit was applied), the next sync includes a **snapshot**: the files' contents and a **hash**.

Concept: Hash

A **hash** is a short fingerprint computed from content: change one character and the fingerprint changes completely. Comparing two hashes tells you whether two files are identical without comparing them line by line. The laptop only sends a snapshot when the hash differs from the last one sent.

The server stores the latest snapshot. The web app shows your commands from it. It also includes the list of primitives your agent has (Doc 03 §8), so the web app and the AI command builder only offer actions that exist on your laptop.

6.2 Server → laptop: edits wait to be applied

A change made in the web app is *not* applied on the server. It becomes a **pending edit**: "change command `play`: default target `spotify`", recorded together with the snapshot hash it was based on. On the next sync the laptop receives it and decides:

```
for each pending edit:
    if the command it changes is identical to what the web app saw:
        write the change into the YAML file      # preserving your comments
        validate it (Doc 03); if invalid -> undo the write, status = "invalid"
        else status = "applied"                  # hot reload picks it up
    else:
        status = "overridden"                    # you changed it locally: laptop wins
report every status in the next sync
```

The web app shows each edit's status: *waiting for laptop*, *applied*, *invalid*, or *overridden by a change on your laptop*. "Waiting for laptop" does *not* ask you to confirm anything: it only means the edit is stored on the server and the laptop has not collected it yet (it is offline, or the next sync has not happened). The laptop applies it automatically; online, that takes at most about 5 minutes.

Edge case: the file is open in PyCharm. Listen sees only what is saved on disk. If you have *unsaved* changes to the same file when a web edit is written, PyCharm notices the file changed on disk and asks which version to keep. Nothing is lost silently.

Concept: Optimistic concurrency

Rather than locking a command while someone edits it, each edit carries the version it was based on, and is accepted only if that version is still current. It is "optimistic" because it assumes conflicts are rare and only checks at the end. Git does the same when it refuses a push because the remote has moved on.

Why it is done this way

This gives you exactly what you asked for. **Offline**, PyCharm editing always works, because the files are on your laptop. **Web** editing works any time; the change lands within 5 minutes of the laptop being online. And there is never a question of which version is real: the YAML file is.

Writing YAML without destroying it. An ordinary YAML library re-writes the whole file and drops your comments. Listen uses `ruamel.yaml`, which edits in place and keeps comments and layout. A web edit therefore appears in PyCharm and in `git status` as a small, readable change you can review, like any other edit.

7 Who proves it is really your laptop

Concept: Authentication and HTTPS

Authentication is proving who you are. The laptop proves it with a **device token**: a long random secret created once in the web app and stored on the laptop (in the `.env` file from Doc 02, never in git). Every sync request carries it. **HTTPS** encrypts the request on its way, so nobody on the network can read your commands or steal the token.

If the token is ever leaked, you revoke it in the web app and create a new one. The full design of accounts and logins belongs to the server section (Section 6).

8 When things go wrong

Situation	What happens
No internet for a week	Everything works locally; rows wait (never deleted); back-off keeps retries rare; when Windows reports the network is back, Listen syncs at once and uploads the backlog in batches.
Reply lost after the server saved rows	Rows are re-sent; upsert by uuid ignores the duplicates.
Web edit and PyCharm edit to the same command	Laptop wins; web app shows "overridden".
Web edit that makes the file invalid	Write is reverted; web app shows "invalid" with the reason.
Server down	Same as offline.
Laptop clock wrong	Ordering of changes uses the server's cursor, not laptop clocks; laptop times are only labels for display.

Watch out

A privacy guarantee should not depend on remembering it. The sync code has a **test** that fails if it ever reads `continuous_log` or `app_sessions`. If a future change accidentally broke the privacy rule, the test suite would catch it before it ever ran.

9 Cost of all this

One request every 5 minutes is 288 requests a day, mostly a few hundred bytes each: well under 1 MB of traffic per day and a fraction of a second of CPU. The sync thread spends the rest of its life asleep. That is the lightweight target met by design rather than by tuning.

10 Glossary

Authentication
: Proving who you are; here, with a device token.

Cursor
: A bookmark into the server's numbered change log.

Exponential back-off
Waiting longer after each failed attempt.

Hash A short fingerprint of content; any change gives a different hash.

HTTPS
HTTP with encryption, so requests cannot be read or altered on the way.

Idempotent
Doing it twice has the same effect as doing it once.

Optimistic concurrency
Accepting an edit only if the version it was based on is still current.

Pending edit
A web-app change waiting for the laptop to apply it.

Polling
The client repeatedly asking the server for news.

Snapshot
A full copy of the command files at a moment, with their hash.

Tombstone
A "deleted" marker kept until the deletion has been delivered.

Upsert
Insert a row, or update it if it already exists.

11 Check your understanding

Check yourself

1. Why can't the server just send changes to your laptop when they happen?
2. The server stored 200 rows, but the reply was lost. What happens on the next sync, and why are there no duplicates?
3. You delete a command from the review list while offline. Trace what happens over the next two syncs.
4. In the web app you change `play`'s default to Spotify. Meanwhile, offline, you edit the same command in PyCharm. What does the web app show after the next sync?
5. Why does the laptop keep the cursor rather than the server remembering what each laptop has seen?
6. Why is the privacy rule enforced by a test rather than only by careful coding?